

CUADERNILLO DE TRABAJO

Tkinter desde cero

Interfaces gráficas con Python

12 clases · de la primera ventana a un CRUD con listas

Prof. Pereyra

Alumno/a: _____

Curso: _____ Año: _____

Cómo usar este cuadernillo

Este cuadernillo te acompaña durante **12 clases**. Cada clase se apoya en la anterior, así que es importante no saltarse ninguna. Al final vas a haber construido un programa completo con ventanas, botones y una lista de datos que se puede **cargar, ver, modificar y borrar** (eso se llama un **CRUD**).

Cada clase tiene estas partes

Sección	Para qué sirve
 Objetivos	Lo que vas a saber hacer al terminar la clase.
 Teoría	La explicación del tema con ejemplos de código para copiar y probar.
 Para recordar	Un resumen corto de lo más importante.
 ¡Atención!	Errores muy comunes y cómo evitarlos.
 Práctica	Ejercicios del tema. Las estrellas indican la dificultad: ★☆☆ fácil, ★★☆☆ medio, ★★★★★ desafío.
 Mi proyecto	Un paso más de tu proyecto personal, que va creciendo clase a clase.

Lo que necesitás

- Python 3 instalado en la computadora (Tkinter ya viene incluido).
- Un editor: **IDLE** (viene con Python), **Thonny** o **VS Code**.
- Saber lo básico de Python: variables, **print**, **if** y **for**. Las funciones y las listas las repasamos en el camino.

Consejos para trabajar

- Guardá cada ejercicio en un archivo aparte con un nombre claro, por ejemplo `clase3_ej2.py`.
- **Escribí el código a mano**, no lo copies y pegues: así se aprende mucho más rápido.
- Si aparece un error, leé la **última línea** del mensaje en rojo: casi siempre dice qué pasó y en qué línea.
- Probá cambiar cosas de los ejemplos: equivocarse también es aprender.

Recorrido del cuadernillo

Clase	Tema	Proyecto
1	Mi primera ventana	Ventana del proyecto
2	Etiquetas y botones (Label , Button , pack)	Título y botones
3	Eventos: los botones hacen cosas	Botón Salir
4	Ingresar datos con Entry	Primer campo de texto
5	Números, cuentas y errores	Campo numérico
6	Ordenar la ventana con grid	Formulario completo
7	Mensajes y validaciones	Validar el formulario
8	Opciones y marcos	Categorías y marcos
9	Listas de Python y Listbox	Lista en pantalla
10	CRUD I: agregar y listar	Alta y listado
11	CRUD II: modificar y eliminar	Modificar y eliminar
12	CRUD III: buscar, ordenar y cierre	Proyecto final

CLASE

1

Mi primera ventana

🎯 En esta clase vas a aprender

- Qué es una interfaz gráfica y qué es Tkinter.
- Crear una ventana y cambiarle el título, el tamaño y el color.
- Entender para qué sirve `mainloop()`.

 Teoría

Programas de consola y programas con ventanas

Hasta ahora seguramente hiciste programas que funcionan en la **consola**: el programa escribe texto con `print` y vos respondés con `input`. Pero casi todos los programas que usás todos los días (el navegador, un juego, la calculadora) tienen **ventanas, botones y cuadros de texto**. A eso se lo llama **interfaz gráfica** o **GUI** (del inglés *Graphical User Interface*).

Tkinter es una biblioteca que viene incluida con Python y sirve para crear interfaces gráficas. No hace falta instalar nada: solo hay que **importarla**.

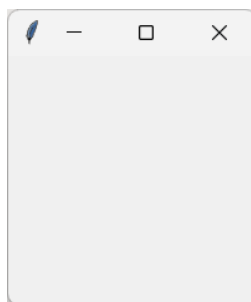
La ventana más simple posible

```
import tkinter as tk      # importamos Tkinter y lo llamamos "tk"

ventana = tk.Tk()         # creamos la ventana principal

ventana.mainloop()       # mantenemos la ventana abierta
```

Así se ve al ejecutarlo:



Veamos qué hace cada línea:

- `import tkinter as tk` trae la biblioteca. Le ponemos el apodo `tk` para escribir menos.
- `tk.Tk()` crea la **ventana principal**. La guardamos en una variable llamada `ventana`.
- `ventana.mainloop()` arranca el **bucle principal**: el programa se queda esperando que el usuario haga algo (un clic, una tecla) hasta que se cierre la ventana. Siempre va **al final**.

Personalizar la ventana

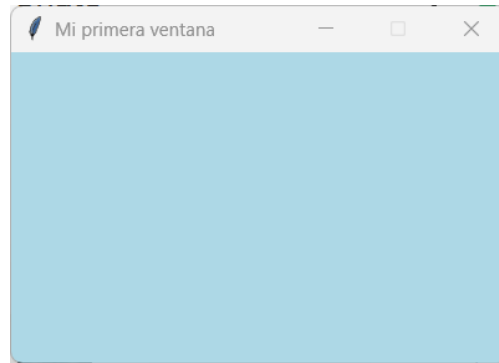
La ventana tiene **métodos** (acciones) que podemos usar para configurarla:

```
import tkinter as tk

ventana = tk.Tk()
ventana.title("Mi primera ventana")    # texto de la barra de título
ventana.geometry("400x250")           # ancho x alto, en píxeles
ventana.config(bg="lightblue")        # color de fondo
ventana.resizable(False, False)        # no se puede agrandar ni achicar

ventana.mainloop()
```

Así se ve al ejecutarlo:



Método	Qué hace	Ejemplo
<code>title()</code>	Cambia el título de la ventana	<code>ventana.title("Hola")</code>
<code>geometry()</code>	Tamaño y, opcionalmente, posición en pantalla	<code>ventana.geometry("400x300+100+50")</code>
<code>config(bg=...)</code>	Color de fondo	<code>ventana.config(bg="yellow")</code>
<code>resizable()</code>	Permite o no cambiar el tamaño (ancho, alto)	<code>ventana.resizable(False, True)</code>

Los colores

Podés escribir los colores de dos formas: **por su nombre en inglés** ("red", "white", "lightgreen", "orange", "gray") o con un **código hexadecimal** que empieza con #, como "#FF0000" (rojo) o "#1F4E79" (azul oscuro). Buscá "selector de color" en internet para encontrar códigos.

💡 Para recordar

- Todo programa de Tkinter empieza con `import tkinter as tk` y `ventana = tk.Tk()`.
- Todo programa de Tkinter termina con `ventana.mainloop()`.
- En `geometry` se usa la letra **x** minúscula, sin espacios: "400x250".

⚠ ¡Atención!

lo que escribas **después** de `mainloop()` recién se ejecuta cuando se cierra la ventana. Por eso siempre va en la última línea.

Práctica

Ejercicio 1.1 ★★★ Creá una ventana con el título "Hola Tkinter" y un tamaño de 300 × 200 píxeles.

Ejercicio 1.2 ★★★ Cambiá el color de fondo de la ventana. Probá con tres colores por nombre y uno con código hexadecimal. Anotá tus favoritos: _____

Ejercicio 1.3 ★★★ Hacé una ventana de 500 × 300 que aparezca en la posición 200, 100 de la pantalla y que **no** se pueda cambiar de tamaño.

Ejercicio 1.4 ★★★ Experimento: agregá la línea `print("Terminó")` después de `mainloop()`. ¿Cuándo aparece el mensaje en la consola? ¿Y qué pasa si borrás `mainloop()` y ejecutás el programa con doble clic? Escribí tu conclusión:

Ejercicio 1.5 ★★★ Creá una ventana que ocupe exactamente la mitad del ancho de tu pantalla. Pista: `ventana.winfo_screenwidth()` te dice el ancho de la pantalla en píxeles.

Mi proyecto: paso a paso

- Elegí el tema de tu proyecto. Durante todo el cuadernillo vas a ir construyendo un programa para administrar una lista de cosas. Ideas: **biblioteca** (libros), **agenda** (contactos), **kiosco** (productos), **colección de videojuegos**, **equipo de fútbol** (jugadores).
- Mi tema es: _____
- Creá el archivo `proyecto.py` con la ventana principal: ponele un título acorde a tu tema, un tamaño cómodo (por ejemplo 500 × 450) y un color de fondo.

CLASE

2

Etiquetas y botones

 En esta clase vas a aprender

- Qué es un **widget**.
- Mostrar texto con **Label**.
- Crear botones con **Button**.
- Ubicar los widgets en la ventana con **pack()**.

 Teoría

¿Qué es un widget?

Un **widget** es cada uno de los elementos que forman una ventana: un texto, un botón, un cuadro para escribir, una lista... Tkinter tiene muchos widgets. Hoy vemos los dos más usados: **Label** (etiqueta, para mostrar texto) y **Button** (botón).

Poner un widget en la ventana siempre lleva **dos pasos**:

- **Crearlo**, indicando dentro de qué ventana va y cómo se ve.
- **Ubicarlo** en la ventana. Hoy usamos **pack()**, que los va acomodando uno debajo del otro.

```
import tkinter as tk

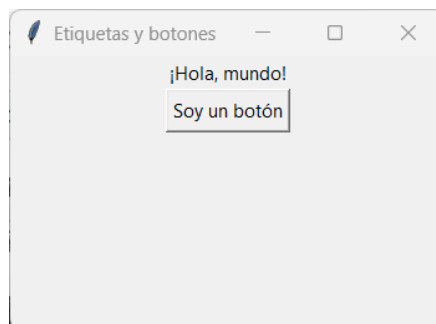
ventana = tk.Tk()
ventana.title("Etiquetas y botones")
ventana.geometry("350x220")

titulo = tk.Label(ventana, text="¡Hola, mundo!") # 1) crear
titulo.pack()                                   # 2) ubicar

boton = tk.Button(ventana, text="Soy un botón")
boton.pack()

ventana.mainloop()
```

Así se ve al ejecutarlo:



⚠ ¡Atención!

si creás un widget pero te olvidás del `pack()`, **no aparece** en la ventana. Es el error más común de esta clase.

Cambiar el aspecto de los widgets

Al crear un widget podemos pasarle **opciones** para cambiar cómo se ve:

Opción	Qué cambia	Ejemplo
<code>text</code>	El texto que muestra	<code>text="Hola"</code>
<code>font</code>	Letra: (nombre, tamaño, estilo)	<code>font=("Arial", 16, "bold")</code>
<code>fg</code>	Color del texto (*foreground*)	<code>fg="white"</code>
<code>bg</code>	Color de fondo (*background*)	<code>bg="navy"</code>
<code>width</code>	Ancho, medido en caracteres	<code>width=15</code>
<code>height</code>	Alto, medido en líneas	<code>height=2</code>

Los estilos de letra pueden ser **"bold"** (negrita), **"italic"** (cursiva) o **"underline"** (subrayado).

Opciones de pack()

`pack()` también acepta opciones para ajustar la ubicación:

- `pady=10 / padx=10`: deja espacio libre arriba y abajo / a los costados.
- `side="left"`: ubica los widgets uno al lado del otro en lugar de uno debajo del otro.
- `fill="x"`: estira el widget para que ocupe todo el ancho.

```
import tkinter as tk

ventana = tk.Tk()
ventana.title("Mi app")
ventana.geometry("400x250")
ventana.config(bg="white")

titulo = tk.Label(ventana, text="Bienvenidos", font=("Arial", 22, "bold"),
                  fg="white", bg="#1F4E79")
titulo.pack(fill="x", pady=(0, 15))

subtitulo = tk.Label(ventana, text="Programación con Python",
                    font=("Arial", 12, "italic"), bg="white")
subtitulo.pack(pady=5)

boton1 = tk.Button(ventana, text="Aceptar", width=10, bg="lightgreen")
boton1.pack(side="left", padx=40, pady=20)

boton2 = tk.Button(ventana, text="Cancelar", width=10, bg="salmon")
boton2.pack(side="left", padx=10, pady=20)

ventana.mainloop()
```

Así se ve al ejecutarlo:



💡 Para recordar

- Un **widget** es cada elemento de la ventana.
- Primero se **crea** (`tk.Label(ventana, ...)`) y después se **ubica** (`.pack()`).
- El primer dato al crear un widget es **dónde va**: **ventana**.
- Por ahora los botones no hacen nada: eso lo vemos la clase que viene.

✎ Práctica

Ejercicio 2.1 ★☆☆ Hacé una ventana con tres etiquetas: tu nombre, tu curso y tu materia favorita.

Ejercicio 2.2 ★☆☆ Mostrá la misma frase en tres etiquetas con diferente letra, tamaño y color cada una.

Ejercicio 2.3 ★☆☆ Poné tres botones en la misma fila ("Nuevo", "Abrir" y "Guardar"), todos del mismo ancho y con espacio entre ellos.

Ejercicio 2.4 ★☆☆ Armá una "tarjeta de presentación": un título grande con fondo de color que ocupe todo el ancho, y abajo tres etiquetas con tus datos.

Ejercicio 2.5 ★☆☆ Dibujá un **semáforo** usando tres etiquetas sin texto (`text=""`), una roja, una amarilla y una verde, con `width=10` y `height=4`, sobre una ventana de fondo negro.

✂ Mi proyecto: paso a paso

- Agregale a tu proyecto un **título grande** con el nombre del programa y un **subtítulo** con tu nombre.
- Agregá dos botones: **"Agregar"** y **"Salir"** (todavía no hacen nada).

CLASE

3

Eventos: los botones hacen cosas

 En esta clase vas a aprender

- Qué es un **evento**.
- Conectar un botón con una **función** usando `command`.
- Cambiar un widget mientras el programa funciona con `config()`.
- Cerrar la ventana desde un botón.

 Teoría

Programar con eventos

En un programa de consola, las instrucciones se ejecutan una detrás de otra, de arriba hacia abajo. En un programa con ventanas es distinto: el programa **espera** a que el usuario haga algo. Cada cosa que hace el usuario (un clic, apretar una tecla, mover el mouse) es un **evento**, y nosotros decidimos qué código se ejecuta cuando ocurre.

Repaso: funciones

Una **función** es un bloque de código con nombre que se ejecuta cuando la llamamos. Se crea con `def`:

```
def saludar():
    print("¡Hola!")

saludar()      # acá la llamamos y se ejecuta
```

La opción `command`

Para que un botón haga algo, le decimos **qué función ejecutar** con la opción `command`:

```
import tkinter as tk

def cambiar_texto():
    mensaje.config(text="¡Hiciste clic!", fg="green")

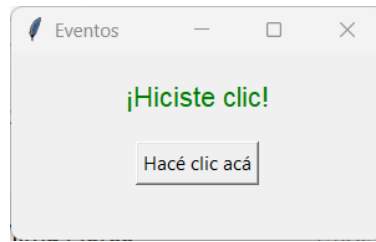
ventana = tk.Tk()
ventana.title("Eventos")
ventana.geometry("300x150")

mensaje = tk.Label(ventana, text="Todavía no hiciste clic", font=("Arial", 13))
mensaje.pack(pady=20)

boton = tk.Button(ventana, text="Hacé clic acá", command=cambiar_texto)
boton.pack()

ventana.mainloop()
```

Así se ve al ejecutarlo:



El método `config()` sirve para **cambiar cualquier opción** de un widget que ya existe: el texto, el color, la letra... Funciona igual en etiquetas, botones y en la ventana.

⚠ ¡Atención!

en `command` va el nombre de la función **sin paréntesis**: `command=cambiar_texto`. Si escribís `command=cambiar_texto()`, la función se ejecuta una sola vez al abrir el programa y el botón después no hace nada.

Un contador de clics

Para que el programa **recuerde** un número entre un clic y otro, usamos una variable afuera de la función. Para poder modificarla desde adentro de la función hay que avisarle a Python con la palabra `global`:

```
import tkinter as tk

contador = 0

def sumar():
    global contador          # vamos a modificar la variable de afuera
    contador = contador + 1
    numero.config(text=str(contador))

def reiniciar():
    global contador
    contador = 0
    numero.config(text="0")

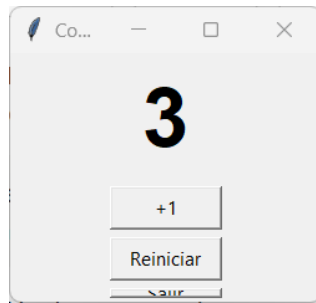
ventana = tk.Tk()
ventana.title("Contador")
ventana.geometry("250x200")

numero = tk.Label(ventana, text="0", font=("Arial", 40, "bold"))
numero.pack(pady=10)

tk.Button(ventana, text="+1", width=10, command=sumar).pack(pady=3)
tk.Button(ventana, text="Reiniciar", width=10, command=reiniciar).pack(pady=3)
tk.Button(ventana, text="Salir", width=10, command=ventana.destroy).pack(pady=3)

ventana.mainloop()
```

Así se ve al ejecutarlo:



Fijate en dos detalles nuevos:

- `str(contador)` convierte el número en texto, porque la opción `text` necesita un texto.
- `ventana.destroy` es una función que ya trae Tkinter y **cierra la ventana**. También va sin paréntesis.
- Si no vamos a volver a usar un botón, podemos crearlo y ubicarlo en una sola línea:
`tk.Button(...).pack()`.

💡 Para recordar

- Un **evento** es algo que hace el usuario (por ejemplo, un clic).
- `command=funcion` conecta un botón con una función, **sin paréntesis**.
- `widget.config(opcion=valor)` cambia un widget que ya existe.
- La función tiene que estar definida **antes** de crear el botón que la usa.

✎ Práctica

Ejercicio 3.1 ★☆☆ Hacé un botón que, al presionarlo, cambie el texto de una etiqueta por tu nombre.

Ejercicio 3.2 ★☆☆ Creá dos botones, "Día" y "Noche": el primero pone el fondo de la ventana en `"lightyellow"` y el segundo en `"midnightblue"`.

Ejercicio 3.3 ★☆☆ Agregale al contador del ejemplo un botón `"-1"` que reste uno.

Ejercicio 3.4 ★☆☆ Hacé un botón que cada vez que lo presiones cambie su **propio** texto a "¡Otra vez!" y su color de fondo.

Ejercicio 3.5 ★★★ Mejorá el contador: que **no pueda bajar de 0**, que el número se ponga **verde** cuando llega a 10 o más, y **rojo** cuando vale 0. Vas a necesitar usar `if`.

✂ Mi proyecto: paso a paso

- Hacé que el botón **"Salir"** de tu proyecto cierre la ventana.
- Agregá una etiqueta vacía abajo de todo. Hacé que al tocar "Agregar" muestre el mensaje "Próximamente...".

CLASE

4

Ingresar datos con Entry

 En esta clase vas a aprender

- Crear cuadros de texto con **Entry**.
- Leer lo que escribió el usuario con **get()**.
- Borrar y escribir texto en un **Entry** desde el código.
- Mover el cursor a un cuadro de texto con **focus()**.

 Teoría

El widget Entry

Un **Entry** es un cuadro donde el usuario puede escribir **una línea de texto**. Es el equivalente al **input()** de la consola, pero en una ventana. Se crea igual que los otros widgets:

```
entrada = tk.Entry(ventana, width=25, font=("Arial", 12))
entrada.pack()
```

Leer lo que escribió el usuario

El método **get()** devuelve el texto que hay escrito en el **Entry** en ese momento. Normalmente se usa dentro de la función de un botón:

```
import tkinter as tk

def saludar():
    nombre = entrada.get()                # leemos el Entry
    resultado.config(text=f"¡Hola, {nombre}!")

ventana = tk.Tk()
ventana.title("Saludador")
ventana.geometry("320x180")

tk.Label(ventana, text="¿Cómo te llamas?").pack(pady=(15, 5))

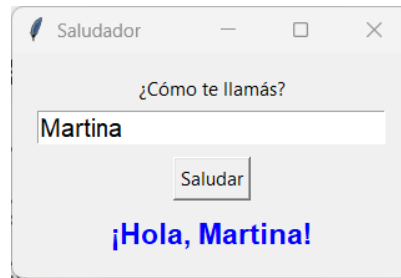
entrada = tk.Entry(ventana, width=25, font=("Arial", 12))
entrada.pack()

tk.Button(ventana, text="Saludar", command=saludar).pack(pady=10)

resultado = tk.Label(ventana, text="", font=("Arial", 14, "bold"), fg="blue")
resultado.pack()

ventana.mainloop()
```

Así se ve al ejecutarlo:



La **f** antes de las comillas indica un **f-string**: todo lo que pongamos entre llaves **{ }** se reemplaza por el valor de esa variable.

Borrar, escribir y enfocar

Método	Qué hace
<code>entrada.get()</code>	Devuelve el texto escrito (siempre es un texto , un str).
<code>entrada.delete(0, tk.END)</code>	Borra todo el contenido: desde la posición 0 hasta el final.
<code>entrada.insert(0, "Hola")</code>	Escribe "Hola" en la posición 0 (al principio).
<code>entrada.focus()</code>	Pone el cursor en ese cuadro, para escribir sin hacer clic.

```
import tkinter as tk

def mostrar():
    nombre = txt_nombre.get().strip()      # strip() quita espacios de más
    apellido = txt_apellido.get().strip()
    resultado.config(text=f"{apellido.upper()}, {nombre}")

def limpiar():
    txt_nombre.delete(0, tk.END)
    txt_apellido.delete(0, tk.END)
    resultado.config(text="")
    txt_nombre.focus()

ventana = tk.Tk()
ventana.title("Nombre completo")
ventana.geometry("300x230")

tk.Label(ventana, text="Nombre:").pack()
txt_nombre = tk.Entry(ventana)
txt_nombre.pack()

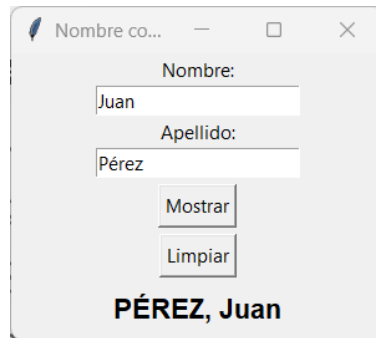
tk.Label(ventana, text="Apellido:").pack()
txt_apellido = tk.Entry(ventana)
txt_apellido.pack()

tk.Button(ventana, text="Mostrar", command=mostrar).pack(pady=5)
tk.Button(ventana, text="Limpiar", command=limpiar).pack()

resultado = tk.Label(ventana, text="", font=("Arial", 13, "bold"))
resultado.pack(pady=10)

txt_nombre.focus()
ventana.mainloop()
```

Así se ve al ejecutarlo:



Algunos métodos útiles de los textos: `.strip()` quita los espacios del principio y del final, `.upper()` pasa todo a mayúsculas, `.lower()` a minúsculas, `.title()` pone la primera letra de cada palabra en mayúscula y `len(texto)` cuenta los caracteres.

💡 Para recordar

- `Entry` sirve para que el usuario escriba una línea de texto.
- `get()` lee, `delete(0, tk.END)` borra, `insert(0, texto)` escribe.
- Lo que devuelve `get()` **siempre es texto**, aunque el usuario escriba números.

✎ Práctica

Ejercicio 4.1 ★☆☆ Hacé el **saludador** del ejemplo y agregale un botón "Limpiar" que borre el cuadro y el saludo.

Ejercicio 4.2 ★☆☆ Pedí una palabra y mostrala en una etiqueta **en mayúsculas**.

Ejercicio 4.3 ★☆☆ Hacé un **contador de letras**: el usuario escribe una frase y el programa muestra cuántos caracteres tiene.

Ejercicio 4.4 ★☆☆ Hacé un programa con un `Entry` y un botón "Copiar": al presionarlo, el texto del cuadro se agrega como una **nueva etiqueta** debajo (cada clic crea una etiqueta más).

Ejercicio 4.5 ★★★★★ Pedí nombre y apellido y generá un **usuario** con la primera letra del nombre + el apellido, todo en minúsculas y sin espacios. Ejemplo: "Juan Pérez" → **jpérez**. Mostralo en un segundo `Entry` usando `insert`.

✂ Mi proyecto: paso a paso

- Agregá a tu proyecto una etiqueta y un `Entry` para el **dato principal** de tu tema (el título del libro, el nombre del contacto, el nombre del producto...).
- Hacé que el botón "Agregar" muestre en la etiqueta de abajo: "Se agregó: ..." con lo que se escribió, y después **limpie** el cuadro.

CLASE

5

Números, cuentas y errores

 En esta clase vas a aprender

- Convertir el texto de un `Entry` en un número con `int()` y `float()`.
- Hacer cálculos con los datos del usuario.
- Evitar que el programa se rompa usando `try / except`.

 TeoríaTodo lo que viene de un `Entry` es texto

Si el usuario escribe `5` en un `Entry`, `get()` devuelve el **texto** `"5"`, no el número `5`. Si intentamos sumar dos textos, Python los **pega**: `"5" + "3"` da `"53"`. Para hacer cuentas hay que **convertir**:

Función	Convierte a...	Ejemplo
<code>int(texto)</code>	Número entero	<code>int("25") → 25</code>
<code>float(texto)</code>	Número con decimales	<code>float("7.5") → 7.5</code>
<code>str(numero)</code>	Texto	<code>str(42) → "42"</code>
<code>round(numero, 2)</code>	Redondea a 2 decimales	<code>round(3.14159, 2) → 3.14</code>

```
import tkinter as tk

def sumar():
    a = float(txt_a.get())
    b = float(txt_b.get())
    resultado.config(text=f"Resultado: {a + b}")

ventana = tk.Tk()
ventana.title("Sumador")
ventana.geometry("260x200")

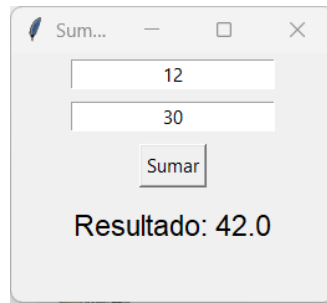
txt_a = tk.Entry(ventana, justify="center")
txt_a.pack(pady=5)
txt_b = tk.Entry(ventana, justify="center")
txt_b.pack(pady=5)

tk.Button(ventana, text="Sumar", command=sumar).pack(pady=5)

resultado = tk.Label(ventana, text="Resultado:", font=("Arial", 14))
resultado.pack(pady=10)

ventana.mainloop()
```

Así se ve al ejecutarlo:



¿Y si el usuario escribe cualquier cosa?

Si en lugar de un número el usuario escribe "hola", `float("hola")` produce un **error** llamado **ValueError** y la función se detiene. Para **atrapar** el error y avisarle al usuario, usamos **try / except**:

```
try:
    # intentamos hacer esto...
    numero = float(txt_a.get())
except ValueError:
    # ...y si falla, hacemos esto otro
    resultado.config(text="Eso no es un número", fg="red")
```

Veamos una calculadora que usa **try / except** y además controla la división por cero:

```
import tkinter as tk

def leer_numeros():
    """Devuelve los dos números, o None si alguno no es válido."""
    try:
        a = float(txt_a.get())
        b = float(txt_b.get())
        return a, b
    except ValueError:
        resultado.config(text="Ingresá dos números válidos", fg="red")
        return None

def calcular(operacion):
    numeros = leer_numeros()
    if numeros is None:
        return # sale de la función sin hacer nada más
    a, b = numeros
    if operacion == "+":
        r = a + b
    elif operacion == "-":
        r = a - b
    elif operacion == "*":
        r = a * b
    else:
        if b == 0:
            resultado.config(text="No se puede dividir por cero", fg="red")
            return
        r = a / b
    resultado.config(text=f"Resultado: {round(r, 2)}", fg="black")

def sumar():
    calcular("+")

def restar():
    calcular("-")

def multiplicar():
```

```

    calcular("*")

def dividir():
    calcular("/")

ventana = tk.Tk()
ventana.title("Calculadora")
ventana.geometry("330x200")

txt_a = tk.Entry(ventana, justify="center")
txt_a.pack(pady=5)
txt_b = tk.Entry(ventana, justify="center")
txt_b.pack(pady=5)

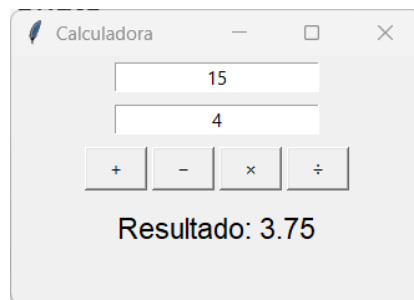
botones = tk.Frame(ventana)          # un contenedor para poner los botones en fila
botones.pack(pady=5)
tk.Button(botones, text="+", width=5, command=sumar).pack(side="left", padx=2)
tk.Button(botones, text="-", width=5, command=restar).pack(side="left", padx=2)
tk.Button(botones, text="x", width=5, command=multiplicar).pack(side="left", padx=2)
tk.Button(botones, text="÷", width=5, command=dividir).pack(side="left", padx=2)

resultado = tk.Label(ventana, text="Resultado:", font=("Arial", 14))
resultado.pack(pady=10)

ventana.mainloop()

```

Así se ve al ejecutarlo:



Dos cosas nuevas: **return** sin valor **corta la función** en ese punto, y **tk.Frame** es un contenedor invisible para agrupar widgets (lo vemos a fondo en la clase 8). Fijate también que la función **leer_numeros** **devuelve** algo con **return**: los números, o **None** (que significa "nada") si hubo un error.

💡 Para recordar

- **get()** devuelve texto: para hacer cuentas usá **int()** o **float()**.
- Si la conversión falla, Python lanza un **ValueError**.
- Con **try / except ValueError**: atrapamos el error y avisamos al usuario en lugar de que el programa falle.

✎ Práctica

Ejercicio 5.1 ★☆☆ Hacé el sumador del ejemplo y agregale **try / except** para que muestre un mensaje si el usuario escribe letras.

Ejercicio 5.2 ★☆☆ Pedí el precio de un producto y mostrá cuánto cuesta con un **descuento del 15 %**.

Ejercicio 5.3 ★★☆☆ Hací un **conversor de temperatura**: el usuario ingresa grados Celsius y el programa muestra los Fahrenheit ($F = C \times 9 / 5 + 32$).

Ejercicio 5.4 ★★☆☆ Pedí tres notas y mostrá el **promedio** redondeado a 2 decimales.

Ejercicio 5.5 ★★★★★ Mejorá el ejercicio 5.4: si el promedio es 6 o más, mostrá "APROBADO" en **verde**; si no, "DESAPROBADO" en **rojo**. Si alguna nota no es un número o no está entre 1 y 10, mostrá un error.

✂ **Mi proyecto: paso a paso**

- Agregá a tu proyecto un segundo campo que sea **numérico** (año de publicación, teléfono, precio, cantidad de goles...).
- Al tocar "Agregar", si ese campo no es un número válido, mostrá un mensaje de error en rojo en la etiqueta de abajo.

CLASE

6

Ordenar la ventana con grid

 En esta clase vas a aprender

- Ubicar widgets en **filas y columnas** con `grid()`.
- Alinear y separar los widgets con `sticky`, `padx` y `pady`.
- Hacer que un widget ocupe varias columnas con `columnspan`.

 Teoría

La ventana como una tabla

Con `pack()` los widgets se acomodan uno debajo del otro (o uno al lado del otro). Para armar **formularios** es mucho más cómodo `grid()`: imaginá la ventana como una **tabla** e indicá en qué **fila** (`row`) y en qué **columna** (`column`) va cada widget. Las filas y columnas se numeran **desde 0**.

	column=0	column=1
row=0	Label "Nombre:"	Entry del nombre
row=1	Label "Apellido:"	Entry del apellido
row=2	Label "Edad:"	Entry de la edad
row=3	Botón "Guardar" (ocupa las dos columnas: <code>columnspan=2</code>)	

```
import tkinter as tk

def guardar():
    nombre = txt_nombre.get()
    apellido = txt_apellido.get()
    edad = txt_edad.get()
    resultado.config(text=f"{nombre} {apellido}, {edad} años")

ventana = tk.Tk()
ventana.title("Formulario")
ventana.geometry("300x200")

tk.Label(ventana, text="Nombre:").grid(row=0, column=0, padx=5, pady=5, sticky="e")
txt_nombre = tk.Entry(ventana)
txt_nombre.grid(row=0, column=1, padx=5, pady=5)

tk.Label(ventana, text="Apellido:").grid(row=1, column=0, padx=5, pady=5, sticky="e")
txt_apellido = tk.Entry(ventana)
txt_apellido.grid(row=1, column=1, padx=5, pady=5)

tk.Label(ventana, text="Edad:").grid(row=2, column=0, padx=5, pady=5, sticky="e")
txt_edad = tk.Entry(ventana)
txt_edad.grid(row=2, column=1, padx=5, pady=5)

tk.Button(ventana, text="Guardar", command=guardar).grid(row=3, column=0,
                                                         columnspan=2, pady=10)
```

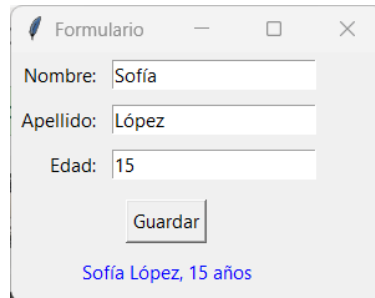
```

resultado = tk.Label(ventana, text="", fg="blue")
resultado.grid(row=4, column=0, columnspan=2)

ventana.mainloop()

```

Así se ve al ejecutarlo:



Opciones de grid()

Opción	Qué hace
<code>row, column</code>	Fila y columna donde va el widget (empiezan en 0).
<code>padx, pady</code>	Espacio libre alrededor del widget, en píxeles.
<code>sticky</code>	Hacia dónde se "pega" el widget dentro de su celda: <code>"n"</code> (arriba), <code>"s"</code> (abajo), <code>"e"</code> (derecha), <code>"w"</code> (izquierda). Se pueden combinar: <code>"ew"</code> lo estira a lo ancho.
<code>columnspan</code>	Cantidad de columnas que ocupa el widget.
<code>rowspan</code>	Cantidad de filas que ocupa el widget.

Como `grid()` usa números, podemos ubicar muchos widgets con un `for`. Por ejemplo, un teclado numérico. Con `//` obtenemos la división entera y con `%` el resto:

```

import tkinter as tk

ventana = tk.Tk()
ventana.title("Teclado")

for i in range(9):
    fila = i // 3          # 0,0,0,1,1,1,2,2,2
    columna = i % 3        # 0,1,2,0,1,2,0,1,2
    boton = tk.Button(ventana, text=str(i + 1), width=5, height=2)
    boton.grid(row=fila, column=columna, padx=2, pady=2)

tk.Button(ventana, text="0", height=2).grid(row=3, column=0, columnspan=3,
                                             sticky="ew", padx=2, pady=2)

ventana.mainloop()

```

Así se ve al ejecutarlo:



⚠ ¡Atención!

no mezcles `pack()` y `grid()` en la misma ventana (o en el mismo **Frame**). Si lo hacés, el programa se queda colgado o muestra un error. Elegí uno para cada contenedor.

💡 Para recordar

- `grid(row=..., column=...)` ubica el widget en una celda de una tabla imaginaria.
- `sticky="e"` alinea a la derecha, `"w"` a la izquierda.
- `columnspan=2` hace que un widget ocupe dos columnas.

✎ Práctica

Ejercicio 6.1 ★☆☆ Rehacé el formulario del ejemplo agregando un campo más: **"Curso"**.

Ejercicio 6.2 ★☆☆ Rehacé la calculadora de la clase 5 usando **solo `grid()`**: los dos cuadros en la fila 0 y 1, los cuatro botones en la fila 2 (uno en cada columna) y el resultado en la fila 3 ocupando las cuatro columnas.

Ejercicio 6.3 ★☆☆ Hacé un formulario de **inscripción** con: nombre, apellido, DNI, fecha de nacimiento y email. Un botón "Inscribir" que muestre un resumen de los datos abajo.

Ejercicio 6.4 ★★★★★ Completá el teclado numérico del ejemplo: agregá un **Entry** arriba de todo (ocupando las 3 columnas) y hacé que cada botón escriba su número en ese **Entry**. Pista: para que cada botón sepa qué número escribir, podés usar `command=lambda n=i+1: escribir(n)` y definir `def escribir(n): pantalla.insert(tk.END, str(n))`.

✂ Mi proyecto: paso a paso

- Reorganizá tu proyecto con `grid()`: armá un **formulario** con al menos **tres campos** con su etiqueta a la izquierda (por ejemplo: título, autor y año).
- Los botones "Agregar" y "Salir" van en una fila debajo del formulario.

CLASE

7

Mensajes y validaciones

 En esta clase vas a aprender

- Mostrar ventanas de mensaje con `messagebox`.
- Hacer preguntas de Sí / No al usuario.
- **Validar** los datos antes de usarlos.

 Teoría

Ventanas de mensaje

Muchas veces queremos avisarle algo al usuario con una ventanita emergente. Para eso, Tkinter tiene el módulo `messagebox`, que hay que importar aparte:

```
from tkinter import messagebox
```

Función	Para qué se usa	Ícono
<code>messagebox.showinfo(título, mensaje)</code>	Informar algo	
<code>messagebox.showwarning(título, mensaje)</code>	Advertir de algo	
<code>messagebox.showerror(título, mensaje)</code>	Avisar de un error	
<code>messagebox.askyesno(título, pregunta)</code>	Preguntar Sí / No. Devuelve <code>True</code> o <code>False</code>	

Validar datos

Validar es comprobar que los datos son correctos **antes** de usarlos: que no haya campos vacíos, que los números sean números, que estén dentro de un rango lógico... La idea es revisar cada condición y, si algo está mal, avisar y **cortar la función con `return`**. Si se llega al final, es porque todo estaba bien.

```
import tkinter as tk
from tkinter import messagebox

def registrar():
    nombre = txt_nombre.get().strip()
    edad_texto = txt_edad.get().strip()

    if nombre == "":
        messagebox.showwarning("Faltan datos", "Escribí tu nombre")
        txt_nombre.focus()
        return
    try:
        edad = int(edad_texto)
    except ValueError:
        messagebox.showerror("Error", "La edad tiene que ser un número entero")
        txt_edad.focus()
    return
```

```

if edad < 5 or edad > 99:
    messagebox.showerror("Error", "La edad tiene que estar entre 5 y 99")
    return

# si llegamos acá, todos los datos son correctos
messagebox.showinfo("Listo", f"Se registró a {nombre} ({edad} años)")

def salir():
    if messagebox.askyesno("Salir", "¿Seguro que querés salir?"):
        ventana.destroy()

ventana = tk.Tk()
ventana.title("Registro")
ventana.geometry("280x150")

tk.Label(ventana, text="Nombre:").grid(row=0, column=0, padx=5, pady=5, sticky="e")
txt_nombre = tk.Entry(ventana)
txt_nombre.grid(row=0, column=1, padx=5, pady=5)

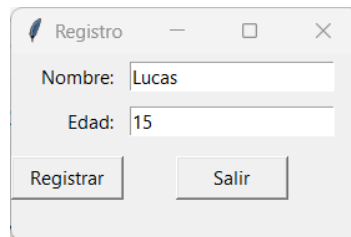
tk.Label(ventana, text="Edad:").grid(row=1, column=0, padx=5, pady=5, sticky="e")
txt_edad = tk.Entry(ventana)
txt_edad.grid(row=1, column=1, padx=5, pady=5)

tk.Button(ventana, text="Registrar", width=10, command=registrar).grid(row=2, column=0,
pady=10)
tk.Button(ventana, text="Salir", width=10, command=salir).grid(row=2, column=1, pady=10)

ventana.mainloop()

```

Así se ve al ejecutarlo:



Un truco más: la opción `show="*"` hace que un `Entry` muestre asteriscos en lugar de las letras, ideal para contraseñas: `tk.Entry(ventana, show="*")`.

💡 Para recordar

- Importá `messagebox` con `from tkinter import messagebox`.
- `askyesno` devuelve `True` si el usuario eligió "Sí": se usa dentro de un `if`.
- Para validar: revisá cada condición, y si algo falla, **avisá y hacé `return`**.

✎ Práctica

Ejercicio 7.1 ★☆☆ Agregale a cualquier programa que hayas hecho un botón "Acerca de" que muestre con `showinfo` tu nombre y la fecha.

Ejercicio 7.2 ★☆☆ Hacé una ventana de **inicio de sesión** con usuario y contraseña (con `show="*"`). Si el usuario es "admin" y la clave "1234", mostrá "Bienvenido"; si no, un error. Nota: esto es solo para practicar; los programas de verdad nunca guardan contraseñas escritas en el código.

Ejercicio 7.3 ★★☆☆ Retomá el ejercicio 5.5 (promedio de notas) y cambiá los mensajes de error de la etiqueta por ventanas de `messagebox`.

Ejercicio 7.4 ★★★ Validá el formulario de inscripción del ejercicio 6.3: todos los campos son obligatorios, el DNI tiene que tener solo números (`texto.isdigit()`) y entre 7 y 8 cifras, y el email tiene que contener una `@`. Al salir, pedí confirmación.

✂ **Mi proyecto: paso a paso**

- Validá el formulario de tu proyecto: los campos obligatorios no pueden estar vacíos y el campo numérico tiene que ser un número (y estar en un rango lógico).
- Mostrá los errores con `messagebox` y un `showinfo` cuando todo esté bien.
- Hacé que el botón "Salir" pida confirmación.

CLASE

8

Opciones y marcos

 En esta clase vas a aprender

- Usar **variables de control**: `StringVar`, `IntVar`, `BooleanVar`.
- Elegir opciones con `Radiobutton` y `Checkbutton`.
- Organizar la ventana en partes con `Frame` y `LabelFrame`.

 Teoría

Variables de control

Son variables especiales de Tkinter que se **conectan** con un widget: si el usuario cambia el widget, la variable cambia sola, y si cambiamos la variable, el widget se actualiza. Se leen con `.get()` y se cambian con `.set()`.

Tipo	Guarda	Ejemplo
<code>tk.StringVar()</code>	Texto	<code>curso = tk.StringVar(value="4° A")</code>
<code>tk.IntVar()</code>	Número entero	<code>cantidad = tk.IntVar(value=0)</code>
<code>tk.BooleanVar()</code>	Verdadero o falso	<code>acepta = tk.BooleanVar()</code>

Radiobutton: elegir UNA opción

Los `Radiobutton` sirven para elegir **una sola opción** entre varias. Todos los de un mismo grupo comparten **la misma variable**, y cada uno tiene su propio **value**: cuando el usuario elige uno, la variable toma ese valor.

Checkbutton: casillas de sí o no

Cada `Checkbutton` es una casilla independiente, con su **propia variable** (normalmente `BooleanVar`), que vale `True` si está marcada.

Frame y LabelFrame: organizar en partes

Un `Frame` es un contenedor invisible donde podemos poner otros widgets. Un `LabelFrame` es igual, pero con un borde y un título. Sirven para dividir la ventana en zonas, y **cada contenedor puede usar su propio pack o grid** sin mezclarse con los demás.

```
import tkinter as tk
from tkinter import messagebox

def enviar():
    extras = []
    if var_musica.get():
        extras.append("Música")
    if var_deporte.get():
        extras.append("Deporte")
    if len(extras) == 0:
```

```

        texto_extras = "ninguna"
    else:
        texto_extras = ", ".join(extras)
    messagebox.showinfo("Inscripción",
                        f"Turno: {var_turno.get()}\nActividades: {texto_extras}")

ventana = tk.Tk()
ventana.title("Inscripción a talleres")

var_turno = tk.StringVar(value="Mañana")
var_musica = tk.BooleanVar()
var_deporte = tk.BooleanVar()

marco_turno = tk.LabelFrame(ventana, text="Turno", padx=10, pady=5)
marco_turno.pack(fill="x", padx=10, pady=5)
tk.Radiobutton(marco_turno, text="Mañana", variable=var_turno,
               value="Mañana").pack(anchor="w")
tk.Radiobutton(marco_turno, text="Tarde", variable=var_turno,
               value="Tarde").pack(anchor="w")
tk.Radiobutton(marco_turno, text="Noche", variable=var_turno,
               value="Noche").pack(anchor="w")

marco_extra = tk.LabelFrame(ventana, text="Actividades", padx=10, pady=5)
marco_extra.pack(fill="x", padx=10, pady=5)
tk.Checkbutton(marco_extra, text="Música", variable=var_musica).pack(anchor="w")
tk.Checkbutton(marco_extra, text="Deporte", variable=var_deporte).pack(anchor="w")

tk.Button(ventana, text="Enviar", width=12, command=enviar).pack(pady=10)

ventana.mainloop()

```

Así se ve al ejecutarlo:



Detalles nuevos: `anchor="w"` alinea el widget a la izquierda dentro del `pack`; `\n` dentro de un texto es un **salto de línea**; y `", ".join(lista)` une los elementos de una lista separados por coma.

Entry con variable de control

Un `Entry` también se puede conectar a una `StringVar` con la opción `textvariable`. Así, para leer o limpiar el cuadro alcanza con `.get()` y `.set("")`:

```

var_nombre = tk.StringVar()
tk.Entry(ventana, textvariable=var_nombre).pack()

print(var_nombre.get()) # leer
var_nombre.set("")      # borrar

```

 Para recordar

- Los **Radiobutton** de un mismo grupo comparten **una** variable; cada uno tiene un **value** distinto.
- Cada **Checkbutton** tiene **su propia** variable.
- Dentro de cada **Frame** podés usar **pack** o **grid**, sin importar lo que use la ventana.

 Práctica

Ejercicio 8.1 ★☆☆ Hacé una ventana para elegir tu **materia favorita** entre cuatro opciones con **Radiobutton**, y un botón que la muestre en una etiqueta.

Ejercicio 8.2 ★☆☆ Hacé un **pedido de pizza**: tamaño (chica \$6000, mediana \$8000, grande \$10000) con **Radiobutton** y agregados (jamón, morrones, aceitunas: \$1500 cada uno) con **Checkbutton**. Un botón muestra el **precio total**.

Ejercicio 8.3 ★☆☆ Reorganizá el formulario del ejercicio 6.3 en dos marcos: un **LabelFrame** "Datos personales" (con **grid**) y un **Frame** abajo con los botones (con **pack**).

Ejercicio 8.4 ★☆☆ Hacé un **Entry** conectado a una **StringVar** y tres botones: "MAYÚSCULAS", "minúsculas" y "Título", que transformen el texto **dentro del mismo cuadro** usando **get()** y **set()**.

 Mi proyecto: paso a paso

- Dividí tu ventana en marcos: un **LabelFrame** con el formulario y un **Frame** con los botones.
- Agregá un dato que se elija con **Radiobutton** (género del libro, tipo de contacto, rubro del producto...).

CLASE

9

Listas de Python y Listbox

 En esta clase vas a aprender

- Repasar las **listas** de Python: crear, agregar, recorrer, modificar y borrar.
- Mostrar elementos en pantalla con el widget **Listbox**.
- Saber qué elemento eligió el usuario con **curselection()**.

 Teoría

Repaso: listas en Python

Una **lista** guarda varios datos en una sola variable, en orden. Cada elemento tiene una **posición** (o índice) que empieza en **0**.

```
materias = ["Matemática", "Historia", "Programación"]

print(materias[0])      # Matemática (el primero es el 0)
print(len(materias))    # 3 (cantidad de elementos)

materias.append("Inglés") # agrega al final
materias[1] = "Geografía" # reemplaza el elemento de la posición 1
materias.pop(0)          # borra el elemento de la posición 0

for materia in materias: # recorre la lista
    print(materia)
```

Operación	Código	Letra del CRUD
Agregar al final	<code>lista.append(dato)</code>	C (crear)
Ver un elemento	<code>lista[i]</code>	R (leer)
Recorrer todos	<code>for x in lista:</code>	R (leer)
Reemplazar	<code>lista[i] = nuevo</code>	U (actualizar)
Borrar por posición	<code>lista.pop(i)</code>	D (borrar)
Cantidad	<code>len(lista)</code>	

El widget Listbox

El **Listbox** muestra una lista de textos en pantalla, uno por renglón, y permite que el usuario **seleccione** uno con un clic. Sus métodos se parecen mucho a los del **Entry**:

Método	Qué hace
<code>lista.insert(tk.END, texto)</code>	Agrega un renglón al final.
<code>lista.delete(i)</code>	Borra el renglón de la posición i .
<code>lista.delete(0, tk.END)</code>	Borra todos los renglones.

Método	Qué hace
<code>lista.curselection()</code>	Devuelve las posiciones seleccionadas, por ejemplo <code>(2,)</code> . Si no hay nada seleccionado devuelve <code>()</code> .
<code>lista.get(i)</code>	Devuelve el texto del renglón <code>i</code> .

La idea clave: dos listas que van juntas

Vamos a tener **una lista de Python** con los datos y un **Listbox** que los muestra. Las posiciones coinciden: el elemento 0 de la lista es el renglón 0 del **Listbox**, el 1 es el 1, y así. Por eso, cuando el usuario selecciona el renglón 2, sabemos que se refiere a `lista[2]`.

```
import tkinter as tk
from tkinter import messagebox

tarefas = []      # la lista de Python con los datos

def agregar():
    texto = txt_tarea.get().strip()
    if texto == "":
        messagebox.showwarning("Atención", "Escribí una tarea")
        return
    tareas.append(texto)          # 1) la agregamos a la lista de Python
    lst_tareas.insert(tk.END, texto) # 2) y también al Listbox
    txt_tarea.delete(0, tk.END)
    txt_tarea.focus()

def eliminar():
    seleccion = lst_tareas.curselection()
    if len(seleccion) == 0:
        messagebox.showwarning("Atención", "Seleccioná una tarea de la lista")
        return
    i = seleccion[0]             # la posición elegida
    tareas.pop(i)                # 1) la borramos de la lista de Python
    lst_tareas.delete(i)         # 2) y del Listbox

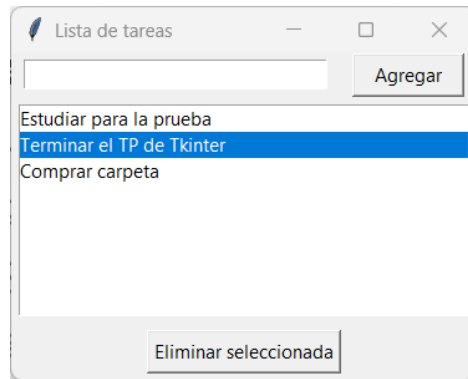
ventana = tk.Tk()
ventana.title("Lista de tareas")

txt_tarea = tk.Entry(ventana, width=30)
txt_tarea.grid(row=0, column=0, padx=5, pady=5)
tk.Button(ventana, text="Agregar", width=10, command=agregar).grid(row=0, column=1,
padx=5)

lst_tareas = tk.Listbox(ventana, width=45, height=8)
lst_tareas.grid(row=1, column=0, columnspan=2, padx=5, pady=5)

tk.Button(ventana, text="Eliminar seleccionada", command=eliminar).grid(row=2, column=0,
columnspan=2,
pady=5)
ventana.mainloop()
```

Así se ve al ejecutarlo:



💡 Para recordar

- Las listas empiezan en la posición **0**.
- `curselection()` devuelve una **tupla** con las posiciones; la primera es `seleccion[0]`.
- Siempre que cambies la lista de Python, hacé el mismo cambio en el **Listbox** para que coincidan.

Práctica

Ejercicio 9.1 ★☆☆ **En consola** (sin ventana): creá una lista con 5 películas, mostralas numeradas con un `for`, cambiá la tercera por otra y borrá la última. Mostrá la lista al final.

Ejercicio 9.2 ★☆☆ Hacé el ejemplo de la lista de tareas y probalo: agregá cinco tareas y eliminá la segunda.

Ejercicio 9.3 ★☆☆ Agregale una etiqueta que muestre **cuántas tareas** hay (`len(tareas)`) y que se actualice cada vez que se agrega o se elimina una.

Ejercicio 9.4 ★☆☆ Agregale un botón "**Vaciar todo**" que, después de pedir confirmación con `askyesno`, borre todas las tareas. Pista: `tareas.clear()` vacía una lista.

Ejercicio 9.5 ★☆☆ Hacé que no se pueda agregar una tarea **repetida**. Pista: `if texto in tareas`: comprueba si el texto ya está en la lista.

✂ Mi proyecto: paso a paso

- Creá en tu proyecto una lista vacía de Python (por ejemplo `libros = []`) y un **Listbox** debajo del formulario.
- Hacé que "Agregar" guarde el **dato principal** en la lista y lo muestre en el **Listbox**.

CLASE

10

CRUD I: agregar y listar

 En esta clase vas a aprender

- Qué es un **CRUD**.
- Guardar registros con varios datos usando **listas dentro de una lista**.
- Mantener el **Listbox** actualizado con una función **refrescar()**.

 Teoría

¿Qué es un CRUD?

Casi todos los programas que manejan información (el sistema de notas de la escuela, la agenda del celular, el inventario de un negocio) hacen las mismas cuatro operaciones. Por sus iniciales en inglés se las llama **CRUD**:

Letra	En inglés	En castellano	En nuestro programa
C	Create	Crear / Alta	Botón "Agregar" (clase 10)
R	Read	Leer / Consultar	El Listbox con todos los datos (clase 10)
U	Update	Actualizar / Modificar	Botón "Modificar" (clase 11)
D	Delete	Borrar / Baja	Botón "Eliminar" (clase 11)

Durante las próximas tres clases vamos a construir un **CRUD de alumnos**: cada alumno tiene **nombre**, **apellido** y **nota**. Vos, en paralelo, vas a hacer lo mismo con el tema de tu proyecto.

Listas dentro de una lista

Cada alumno tiene tres datos, así que lo guardamos como una **lista de tres elementos**. Y todos los alumnos juntos van en **otra lista**:

```
alumno = ["Ana", "García", 8.0]      # un alumno: [nombre, apellido, nota]

alumnos = [
    ["Ana", "García", 8.0],
    ["Luis", "Martínez", 6.5],
    ["Sofía", "López", 9.0],
]

print(alumnos[1])      # ['Luis', 'Martínez', 6.5] -> el segundo alumno
print(alumnos[1][0])   # Luis      -> el nombre del segundo alumno
print(alumnos[2][2])   # 9.0       -> la nota del tercer alumno
```

El primer número entre corchetes dice **qué alumno** y el segundo **qué dato** de ese alumno: **[0]** nombre, **[1]** apellido, **[2]** nota.

La función refrescar()

En la clase anterior, cada vez que cambiábamos la lista de Python teníamos que hacer lo mismo en el **Listbox**. Cuando el programa crece, eso trae errores. Una forma más segura es pensar así:

- **La lista de Python es la que manda:** ahí están los datos de verdad.
- El **Listbox** solo **muestra** esa lista. Cada vez que la lista cambia, llamamos a **refrescar()**, que **borra todo el Listbox y lo vuelve a llenar** a partir de la lista.

El programa: alta y listado

```
import tkinter as tk
from tkinter import messagebox

# ----- DATOS -----
alumnos = []          # cada alumno es una lista: [nombre, apellido, nota]

# ----- FUNCIONES -----
def refrescar():
    """Vuelve a llenar el Listbox a partir de la lista alumnos."""
    lst_alumnos.delete(0, tk.END)
    for alumno in alumnos:
        lst_alumnos.insert(tk.END, f"{alumno[1]}, {alumno[0]} - Nota: {alumno[2]}")
    lbl_total.config(text=f"Total: {len(alumnos)} alumnos")

def limpiar():
    txt_nombre.delete(0, tk.END)
    txt_apellido.delete(0, tk.END)
    txt_nota.delete(0, tk.END)
    txt_nombre.focus()

def agregar():
    nombre = txt_nombre.get().strip()
    apellido = txt_apellido.get().strip()
    if nombre == "" or apellido == "":
        messagebox.showwarning("Faltan datos", "Completa nombre y apellido")
        return
    try:
        nota = float(txt_nota.get())
    except ValueError:
        messagebox.showerror("Error", "La nota tiene que ser un número")
        return
    if nota < 1 or nota > 10:
        messagebox.showerror("Error", "La nota tiene que estar entre 1 y 10")
        return
    alumnos.append([nombre, apellido, nota])    # C: crear
    refrescar()                                # R: mostrar
    limpiar()

# ----- VENTANA -----
ventana = tk.Tk()
ventana.title("Gestión de alumnos")

marco_datos = tk.LabelFrame(ventana, text="Datos del alumno", padx=10, pady=10)
marco_datos.pack(fill="x", padx=10, pady=10)

tk.Label(marco_datos, text="Nombre:").grid(row=0, column=0, sticky="e")
txt_nombre = tk.Entry(marco_datos, width=30)
txt_nombre.grid(row=0, column=1, pady=3)

tk.Label(marco_datos, text="Apellido:").grid(row=1, column=0, sticky="e")
txt_apellido = tk.Entry(marco_datos, width=30)
txt_apellido.grid(row=1, column=1, pady=3)
```

```

tk.Label(marco_datos, text="Nota:").grid(row=2, column=0, sticky="e")
txt_nota = tk.Entry(marco_datos, width=30)
txt_nota.grid(row=2, column=1, pady=3)

tk.Button(ventana, text="Agregar", width=12, command=agregar).pack()

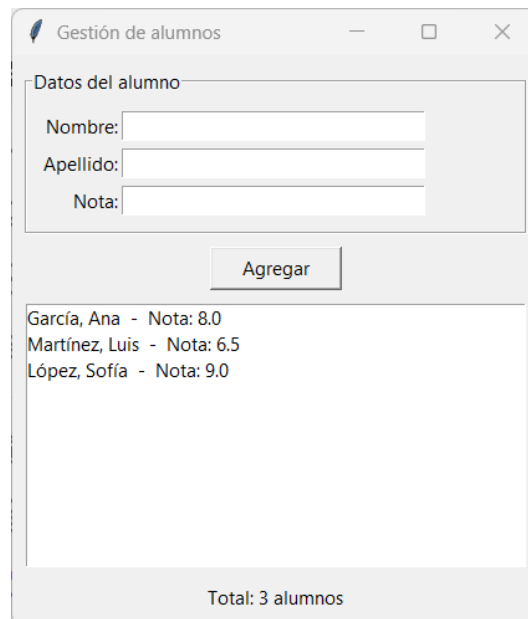
lst_alumnos = tk.Listbox(ventana, width=50, height=10)
lst_alumnos.pack(padx=10, pady=10)

lbl_total = tk.Label(ventana, text="Total: 0 alumnos")
lbl_total.pack(pady=(0, 10))

txt_nombre.focus()
ventana.mainloop()

```

Así se ve al ejecutarlo:



Fijate en cómo está **ordenado** el programa: primero los **datos**, después las **funciones** y al final la **ventana**. Mantener ese orden hace que el código sea mucho más fácil de leer y de corregir.

💡 Para recordar

- **CRUD** = Crear, Leer, Actualizar y Borrar.
- Cada registro es una lista; todos los registros van en otra lista: `alumnos[i][dato]`.
- La lista de Python manda; el **Listbox** solo la muestra. Después de cada cambio, llamá a `refrescar()`.

✎ Práctica

Ejercicio 10.1 ★☆☆ Escribí el programa del ejemplo, ejecutalo y cargá cinco alumnos. Probá dejar campos vacíos y escribir letras en la nota.

Ejercicio 10.2 ★☆☆ Hacé que el programa arranque con **tres alumnos de ejemplo** ya cargados. Pista: cargalos en la lista `alumnos` y llamá a `refrescar()` justo antes de `mainloop()`.

Ejercicio 10.3 ★☆☆ Agregá un cuarto dato: el **curso** del alumno. Tenés que cambiar el formulario, la validación, el `append` y el texto que arma `refrescar()`.

Ejercicio 10.4 ★★★ Hacé que la etiqueta de abajo muestre también el **promedio general** del curso. Cuidado: si la lista está vacía no se puede dividir por cero.

✂ **Mi proyecto: paso a paso**

- Transformá tu proyecto en la **C** y la **R** de un CRUD: cada registro tiene que ser una lista con al menos **tres datos** (uno de ellos numérico).
- Usá una función **refrescar()** y mostrá el total de registros.

CLASE

11

CRUD II: modificar y eliminar

 En esta clase vas a aprender

- Detectar cuando el usuario **selecciona** un renglón del **Listbox**.
- Cargar los datos seleccionados en el formulario.
- **Modificar** (U) y **eliminar** (D) registros.

 TeoríaReaccionar a la selección con `bind()`

Hasta ahora solo reaccionábamos a los clics en botones con `command`. Pero podemos conectar una función a **otros eventos** con el método `bind()`. El **Listbox** genera el evento "`<<ListboxSelect>>`" cada vez que el usuario selecciona un renglón:

```
lst_alumnos.bind("<<ListboxSelect>>", al_seleccionar)
```

Las funciones conectadas con `bind()` reciben un dato extra con información del evento, por eso se escriben con un parámetro: `def al_seleccionar(event):`. No hace falta usarlo, pero tiene que estar.

Las funciones nuevas

Partimos del programa de la clase 10. Primero, una función que nos dice **qué alumno está seleccionado**. Si no hay ninguno, avisa y devuelve `None`:

```
def posicion_seleccionada():
    """Devuelve la posición elegida en el Listbox, o None si no hay ninguna."""
    seleccion = lst_alumnos.curselection()
    if len(seleccion) == 0:
        messagebox.showwarning("Atención", "Primero seleccioná un alumno de la lista")
        return None
    return seleccion[0]
```

 ¡Atención!

para preguntar si no hay selección escribimos `if i is None:` y **no** `if not i:`. ¿Por qué? Porque la posición del primer alumno es **0**, y para Python el 0 cuenta como "falso": el programa pensaría que no hay nada seleccionado.

Cuando el usuario elige un alumno, copiamos sus datos al formulario para que pueda editarlos:

```
def al_seleccionar(event):
    seleccion = lst_alumnos.curselection()
    if len(seleccion) == 0:
        return
    alumno = alumnos[seleccion[0]]
    txt_nombre.delete(0, tk.END)
    txt_nombre.insert(0, alumno[0])
    txt_apellido.delete(0, tk.END)
    txt_apellido.insert(0, alumno[1])
    txt_nota.delete(0, tk.END)
    txt_nota.insert(0, str(alumno[2]))
```

Para **modificar**, reemplazamos el alumno de esa posición por los datos nuevos del formulario. Para **eliminar**, pedimos confirmación y lo sacamos con **pop()**. En los dos casos, después **refrescamos**:

```
def modificar():
    i = posicion_seleccionada()
    if i is None:
        return
    datos = leer_formulario()
    if datos is None:
        return
    alumnos[i] = datos                # U: actualizar
    refrescar()
    limpiar()

def eliminar():
    i = posicion_seleccionada()
    if i is None:
        return
    alumno = alumnos[i]
    if messagebox.askyesno("Confirmar", f"¿Eliminar a {alumno[0]} {alumno[1]}?"):
        alumnos.pop(i)                # D: borrar
        refrescar()
        limpiar()
```

Una mejora: leer_formulario()

Tanto "Agregar" como "Modificar" necesitan leer y validar el formulario. En lugar de escribir la validación dos veces, la ponemos en **una sola función** que devuelve la lista **[nombre, apellido, nota]** si todo está bien, o **None** si hay un error. Así, **agregar()** queda cortita:

```
def leer_formulario():
    """Valida el formulario. Devuelve [nombre, apellido, nota] o None."""
    nombre = txt_nombre.get().strip()
    apellido = txt_apellido.get().strip()
    if nombre == "" or apellido == "":
        messagebox.showwarning("Faltan datos", "Completá nombre y apellido")
        return None
    try:
        nota = float(txt_nota.get())
    except ValueError:
        messagebox.showerror("Error", "La nota tiene que ser un número")
        return None
    if nota < 1 or nota > 10:
        messagebox.showerror("Error", "La nota tiene que estar entre 1 y 10")
        return None
    return [nombre, apellido, nota]

def agregar():
    datos = leer_formulario()
    if datos is None:
        return
    alumnos.append(datos)          # C: crear
    refrescar()
    limpiar()
```

Cambios en la ventana

Por último, agregamos los botones nuevos en un **Frame** y conectamos el evento de selección. Hay un detalle importante: al crear el **Listbox** le agregamos **exportselection=False**. Sin esa opción, cuando el usuario hace clic en un **Entry** para editar, el **Listbox** pierde la selección y "Modificar" no sabría qué alumno cambiar.

```
marco_botones = tk.Frame(ventana)
marco_botones.pack()
tk.Button(marco_botones, text="Agregar", width=10,
          command=agregar).pack(side="left", padx=3)
tk.Button(marco_botones, text="Modificar", width=10,
          command=modificar).pack(side="left", padx=3)
tk.Button(marco_botones, text="Eliminar", width=10,
          command=eliminar).pack(side="left", padx=3)
tk.Button(marco_botones, text="Limpiar", width=10,
          command=limpiar).pack(side="left", padx=3)

lst_alumnos = tk.Listbox(ventana, width=50, height=10, exportselection=False)
lst_alumnos.pack(padx=10, pady=10)
lst_alumnos.bind("<<ListboxSelect>>", al_seleccionar)
```

Y en **limpiar()** agregamos una línea para quitar la selección del **Listbox**:

lst_alumnos.selection_clear(0, tk.END). El programa completo, con todo esto integrado, está en la clase 12.

Para recordar

- **bind("<<ListboxSelect>>", funcion)** ejecuta la función cuando el usuario selecciona un renglón.
- **Modificar = lista[i] = nuevos_datos**. **Eliminar = lista.pop(i)**. Siempre seguido de **refrescar()**.
- Usá **exportselection=False** en el **Listbox** para no perder la selección.
- Si una validación se repite, ponela en **una función** y reutilízala.

 Práctica

Ejercicio 11.1 ★☆☆ Agregá al programa de la clase 10 todas las funciones y cambios de esta clase. Probá: agregar, seleccionar, modificar una nota y eliminar un alumno.

Ejercicio 11.2 ★☆☆ Después de eliminar, mostrá un `showinfo` que diga a quién se eliminó. Después de modificar, otro que diga "Datos actualizados".

Ejercicio 11.3 ★☆☆ Hacé que **no se pueda agregar** un alumno con el mismo nombre y apellido que uno que ya existe. Pista: recorré la lista con un `for` y compará `alumno[0]` y `alumno[1]`.

Ejercicio 11.4 ★★★ Hacé que los botones "Modificar" y "Eliminar" empiecen **desactivados** (`state="disabled"`) y se activen (`boton.config(state="normal")`) solo cuando hay un alumno seleccionado. Al limpiar, se vuelven a desactivar.

 Mi proyecto: paso a paso

- Completá el CRUD de tu proyecto: seleccionar un registro carga sus datos en el formulario, y los botones "**Modificar**" y "**Eliminar**" funcionan (con confirmación antes de borrar).
- Creá tu propia función `leer_formulario()` con todas las validaciones.

CLASE

12

CRUD III: buscar, ordenar y cierre

 En esta clase vas a aprender

- **Buscar** un registro en la lista.
- **Ordenar** la lista por distintos criterios.
- Integrar todo en el **programa final** y presentar tu proyecto.

 Teoría

Buscar

Para buscar un alumno por apellido recorreremos la lista con un **for** usando las **posiciones** (`range(len(alumnos))`), porque necesitamos saber **dónde** está. Comparamos todo en minúsculas con `.lower()` para que "garcía" encuentre a "García", y usamos **in** para que alcance con escribir una parte del apellido. Cuando lo encontramos, lo **seleccionamos** en el **Listbox**:

```
def buscar():
    texto = txt_buscar.get().strip().lower()
    if texto == "":
        return
    for i in range(len(alumnos)):
        if texto in alumnos[i][1].lower():
            lst_alumnos.selection_clear(0, tk.END)
            lst_alumnos.selection_set(i)      # selecciona el renglón i
            lst_alumnos.see(i)                # se desplaza para que se vea
            al_seleccionar(None)              # carga sus datos en el formulario
            return
    messagebox.showinfo("Buscar", "No se encontró ningún alumno con ese apellido")
```

Seleccionar un renglón desde el código **no** dispara el evento `<<ListboxSelect>>`. Por eso llamamos nosotros a `al_seleccionar(None)` (le pasamos **None** porque no hay evento real).

Ordenar

Las listas tienen el método `sort()`, que las ordena. Como cada alumno es una lista, tenemos que decirle **por cuál dato** ordenar. Para eso le pasamos en **key** una función que, dado un alumno, devuelve el dato a comparar:

```
def por_apellido(alumno):
    return alumno[1].lower()

def por_nota(alumno):
    return alumno[2]

def ordenar_apellido():
    alumnos.sort(key=por_apellido)
    refrescar()

def ordenar_nota():
    alumnos.sort(key=por_nota, reverse=True)  # reverse=True: de mayor a menor
    refrescar()
```

El programa completo

Este es el CRUD terminado, con todo lo que vimos en las últimas tres clases. Léelo con calma: ya conocés **cada una** de sus partes.

```
import tkinter as tk
from tkinter import messagebox

# ===== DATOS =====
alumnos = []  # cada alumno es una lista: [nombre, apellido, nota]

# ===== FUNCIONES =====
def refrescar():
    lst_alumnos.delete(0, tk.END)
    for alumno in alumnos:
        lst_alumnos.insert(tk.END, f"{alumno[1]}, {alumno[0]} - Nota: {alumno[2]}")
    lbl_total.config(text=f"Total: {len(alumnos)} alumnos")

def limpiar():
    txt_nombre.delete(0, tk.END)
    txt_apellido.delete(0, tk.END)
    txt_nota.delete(0, tk.END)
    lst_alumnos.selection_clear(0, tk.END)
    txt_nombre.focus()

def leer_formulario():
    """Valida el formulario. Devuelve [nombre, apellido, nota] o None."""
    nombre = txt_nombre.get().strip()
    apellido = txt_apellido.get().strip()
    if nombre == "" or apellido == "":
        messagebox.showwarning("Faltan datos", "Completá nombre y apellido")
        return None
    try:
        nota = float(txt_nota.get())
    except ValueError:
        messagebox.showerror("Error", "La nota tiene que ser un número")
        return None
    if nota < 1 or nota > 10:
        messagebox.showerror("Error", "La nota tiene que estar entre 1 y 10")
        return None
    return [nombre, apellido, nota]

def posicion_seleccionada():
    seleccion = lst_alumnos.curselection()
    if len(seleccion) == 0:
        messagebox.showwarning("Atención", "Primero seleccioná un alumno de la lista")
        return None
    return seleccion[0]

def al_seleccionar(event):
```

```

seleccion = lst_alumnos.curselection()
if len(seleccion) == 0:
    return
alumno = alumnos[seleccion[0]]
txt_nombre.delete(0, tk.END)
txt_nombre.insert(0, alumno[0])
txt_apellido.delete(0, tk.END)
txt_apellido.insert(0, alumno[1])
txt_nota.delete(0, tk.END)
txt_nota.insert(0, str(alumno[2]))

# --- C: crear ---
def agregar():
    datos = leer_formulario()
    if datos is None:
        return
    alumnos.append(datos)
    refrescar()
    limpiar()

# --- U: actualizar ---
def modificar():
    i = posicion_seleccionada()
    if i is None:
        return
    datos = leer_formulario()
    if datos is None:
        return
    alumnos[i] = datos
    refrescar()
    limpiar()

# --- D: borrar ---
def eliminar():
    i = posicion_seleccionada()
    if i is None:
        return
    alumno = alumnos[i]
    if messagebox.askyesno("Confirmar", f"¿Eliminar a {alumno[0]} {alumno[1]}?"):
        alumnos.pop(i)
        refrescar()
        limpiar()

# --- Buscar y ordenar ---
def buscar():
    texto = txt_buscar.get().strip().lower()
    if texto == "":
        return
    for i in range(len(alumnos)):
        if texto in alumnos[i][1].lower():
            lst_alumnos.selection_clear(0, tk.END)
            lst_alumnos.selection_set(i)
            lst_alumnos.see(i)
            al_seleccionar(None)
            return
    messagebox.showinfo("Buscar", "No se encontró ningún alumno con ese apellido")

def por_apellido(alumno):
    return alumno[1].lower()

def por_nota(alumno):
    return alumno[2]

def ordenar_apellido():
    alumnos.sort(key=por_apellido)
    refrescar()

```

```

def ordenar_nota():
    alumnos.sort(key=por_nota, reverse=True)
    refrescar()

def salir():
    if messagebox.askyesno("Salir", "¿Seguro que querés salir?"):
        ventana.destroy()

# ===== VENTANA =====
ventana = tk.Tk()
ventana.title("Gestión de alumnos")
ventana.resizable(False, False)

tk.Label(ventana, text="Gestión de alumnos", font=("Arial", 16, "bold"),
        fg="white", bg="#1F4E79").pack(fill="x")

# Formulario
marco_datos = tk.LabelFrame(ventana, text="Datos del alumno", padx=10, pady=10)
marco_datos.pack(fill="x", padx=10, pady=10)

tk.Label(marco_datos, text="Nombre:").grid(row=0, column=0, sticky="e")
txt_nombre = tk.Entry(marco_datos, width=35)
txt_nombre.grid(row=0, column=1, pady=3)

tk.Label(marco_datos, text="Apellido:").grid(row=1, column=0, sticky="e")
txt_apellido = tk.Entry(marco_datos, width=35)
txt_apellido.grid(row=1, column=1, pady=3)

tk.Label(marco_datos, text="Nota:").grid(row=2, column=0, sticky="e")
txt_nota = tk.Entry(marco_datos, width=35)
txt_nota.grid(row=2, column=1, pady=3)

# Botones del CRUD
marco_botones = tk.Frame(ventana)
marco_botones.pack()
tk.Button(marco_botones, text="Agregar", width=10, bg="#C6E0B4",
        command=agregar).pack(side="left", padx=3)
tk.Button(marco_botones, text="Modificar", width=10, bg="#FFE699",
        command=modificar).pack(side="left", padx=3)
tk.Button(marco_botones, text="Eliminar", width=10, bg="#F4B183",
        command=eliminar).pack(side="left", padx=3)
tk.Button(marco_botones, text="Limpiar", width=10,
        command=limpiar).pack(side="left", padx=3)

# Buscar y ordenar
marco_buscar = tk.LabelFrame(ventana, text="Buscar y ordenar", padx=10, pady=5)
marco_buscar.pack(fill="x", padx=10, pady=10)
tk.Label(marco_buscar, text="Apellido:").pack(side="left")
txt_buscar = tk.Entry(marco_buscar, width=15)
txt_buscar.pack(side="left", padx=5)
tk.Button(marco_buscar, text="Buscar", command=buscar).pack(side="left")
tk.Button(marco_buscar, text="Por nota", command=ordenar_nota).pack(side="right")
tk.Button(marco_buscar, text="A-Z", command=ordenar_apellido).pack(side="right", padx=5)

# Listado
lst_alumnos = tk.Listbox(ventana, width=55, height=10, exportselection=False)
lst_alumnos.pack(padx=10)
lst_alumnos.bind("<<ListboxSelect>>", al_seleccionar)

lbl_total = tk.Label(ventana, text="Total: 0 alumnos")
lbl_total.pack(pady=5)

tk.Button(ventana, text="Salir", width=10, command=salir).pack(pady=(0, 10))

```

```
txt_nombre.focus()
ventana.mainloop()
```

Así se ve al ejecutarlo:

💡 Para recordar

- Para buscar, recorré con `for i in range(len(lista))` y compará en minúsculas.
- `selection_set(i)` y `see(i)` seleccionan y muestran un renglón desde el código.
- `lista.sort(key=funcion)` ordena según lo que devuelve la función; `reverse=True` invierte el orden.

✎ Práctica

Ejercicio 12.1 ★★★ Escribí el programa completo y probá **todas** las funciones: agregar, seleccionar, modificar, eliminar, buscar y ordenar.

Ejercicio 12.2 ★★★ Agregá un botón "**Mejor nota**" que muestre con `showinfo` el nombre del alumno con la nota más alta.

Ejercicio 12.3 ★★★ Agregá un botón "**Aprobados**" que muestre en un `showinfo` cuántos alumnos tienen 6 o más y cuántos tienen menos de 6.

Ejercicio 12.4 ★★★ Modificá la búsqueda para que, si hay **varios** alumnos que coinciden, los seleccione a todos. Pista: no hagas `return` al encontrar el primero, y usá una variable para saber si encontraste alguno. (Para seleccionar varios, el `Listbox` necesita la opción `selectmode="multiple"`.)

Proyecto final

Llegó el momento de terminar y presentar el proyecto que venís construyendo desde la clase 1. Tiene que ser un **CRUD completo** sobre el tema que elegiste.

Requisitos obligatorios

- Ventana con título, formulario ordenado con **grid** y marcos (**Frame** / **LabelFrame**).
- Cada registro es una **lista** con al menos **tres datos** (al menos uno numérico), y todos los registros se guardan en otra lista.
- Las cuatro operaciones del CRUD funcionando: **agregar**, **listar**, **modificar** y **eliminar**.
- Validaciones con **messagebox**: campos vacíos, datos numéricos y rangos lógicos.
- Confirmación antes de eliminar y antes de salir.
- Una función de **búsqueda** y al menos un criterio de **ordenamiento**.
- Código ordenado (datos → funciones → ventana), con nombres claros y algunos comentarios.

Ideas para sumar puntos

- Usar **Radiobutton** o **Checkbutton** para algún dato.
- Mostrar estadísticas: total, promedio, máximo, mínimo...
- Desactivar los botones que no se pueden usar en cada momento.
- Precargar algunos registros de ejemplo al iniciar.
- Agregar una barra de desplazamiento (**Scrollbar**) al **Listbox** (investigá cómo se hace).

Cómo se evalúa

Criterio	Qué se mira	Puntos
Interfaz	Ventana ordenada, clara y fácil de usar	2
CRUD	Agregar, listar, modificar y eliminar funcionan correctamente	3
Validaciones	El programa no se rompe con datos incorrectos y avisa al usuario	2
Buscar y ordenar	La búsqueda y el ordenamiento funcionan	1
Código	Ordenado, con buenos nombres y comentarios	1
Presentación	Explicar el funcionamiento y responder preguntas sobre el código	1
Total		10

Autoevaluación: antes de entregar, comprobá que...

- ☐ Puedo agregar un registro y aparece en la lista.
- ☐ Si dejo un campo vacío o escribo letras en un campo numérico, el programa me avisa y no se rompe.
- ☐ Puedo seleccionar un registro y sus datos aparecen en el formulario.
- ☐ Puedo modificar un registro y el cambio se ve en la lista.
- ☐ Puedo eliminar un registro, y antes me pide confirmación.
- ☐ Si toco "Modificar" o "Eliminar" sin seleccionar nada, el programa me avisa.
- ☐ La búsqueda encuentra registros aunque escriba en minúsculas.
- ☐ Puedo explicar qué hace cada función de mi código.

Anexo A: Referencia rápida

Estructura de todo programa

```
import tkinter as tk
from tkinter import messagebox

# 1) Datos
# 2) Funciones
# 3) Ventana y widgets

ventana = tk.Tk()
# ... widgets ...
ventana.mainloop()
```

Widgets

Widget	Para qué sirve	Métodos y opciones más usados
<code>tk.Label</code>	Mostrar texto	<code>text, font, fg, bg, config(text=...)</code>
<code>tk.Button</code>	Ejecutar una acción	<code>text, command, width, state</code>
<code>tk.Entry</code>	Escribir una línea	<code>get(), delete(0, tk.END), insert(0, t), focus(), show=""</code>
<code>tk.Listbox</code>	Mostrar una lista	<code>insert(tk.END, t), delete(i), curselection(), selection_set(i)</code>
<code>tk.Radiobutton</code>	Elegir una opción	<code>variable, value</code>
<code>tk.Checkbutton</code>	Casilla sí / no	<code>variable (BooleanVar)</code>
<code>tk.Frame</code>	Agrupar widgets	<code>padx, pady</code>
<code>tk.LabelFrame</code>	Agrupar con borde y título	<code>text, padx, pady</code>

Ubicar widgets

Método	Opciones útiles
<code>pack()</code>	<code>side="left", fill="x", padx, pady, anchor="w"</code>
<code>grid()</code>	<code>row, column, sticky="e", columnspan, padx, pady</code>

Listas de Python

Quiero...	Código
Crear una lista vacía	<code>datos = []</code>
Agregar al final	<code>datos.append(x)</code>
Leer / reemplazar la posición i	<code>datos[i] / datos[i] = x</code>
Borrar la posición i	<code>datos.pop(i)</code>
Cantidad de elementos	<code>len(datos)</code>
Recorrer con posiciones	<code>for i in range(len(datos)):</code>
Ordenar	<code>datos.sort(key=funcion, reverse=True)</code>
Vaciar	<code>datos.clear()</code>

Anexo B: Errores frecuentes

Cuando el programa falla, Python muestra un mensaje en rojo. **Leé la última línea:** dice el tipo de error. Un poco más arriba aparece **line N**, que indica en qué línea ocurrió.

Síntoma o mensaje	Causa probable	Solución
El widget no aparece	Falta el <code>pack()</code> o el <code>grid()</code>	Ubicá el widget después de crearlo.
El botón no hace nada, o la acción ocurre sola al abrir	Se escribió <code>command=funcion()</code>	Sacá los paréntesis: <code>command=funcion</code> .
<code>NameError: name 'x' is not defined</code>	La variable o función no existe o está mal escrita (ojo con mayúsculas)	Revisá el nombre. Las funciones tienen que estar definidas antes de usarlas.
<code>ValueError: could not convert string to float</code>	Se intentó convertir un texto que no es número	Usá <code>try / except ValueError</code> .
<code>IndexError: list index out of range</code>	Se usó una posición que no existe en la lista	Verificá que haya algo seleccionado y que la lista y el <code>Listbox</code> coincidan.
<code>UnboundLocalError</code>	Se modifica una variable de afuera dentro de una función	Agregá <code>global variable</code> al principio de la función.
<code>TclError: cannot use geometry manager grid inside . which already has slaves managed by pack</code>	Se mezclaron <code>pack</code> y <code>grid</code> en el mismo contenedor	Usá uno solo por contenedor, o separá en <code>Frame</code> .
<code>IndentationError</code>	La sangría (los espacios al principio) está mal	Usá 4 espacios por nivel y no mezcles espacios con tabulaciones.
Al hacer clic en un <code>Entry</code> se pierde la selección de la lista	Comportamiento normal del <code>Listbox</code>	Creá el <code>Listbox</code> con <code>exportselection=False</code> .
Se suman textos en vez de números (<code>"5" + "3" = "53"</code>)	Falta convertir lo que devuelve <code>get()</code>	Usá <code>int()</code> o <code>float()</code> .